A Primer on Graphics and Graphics Hardware

Graphics From A User's Perspective

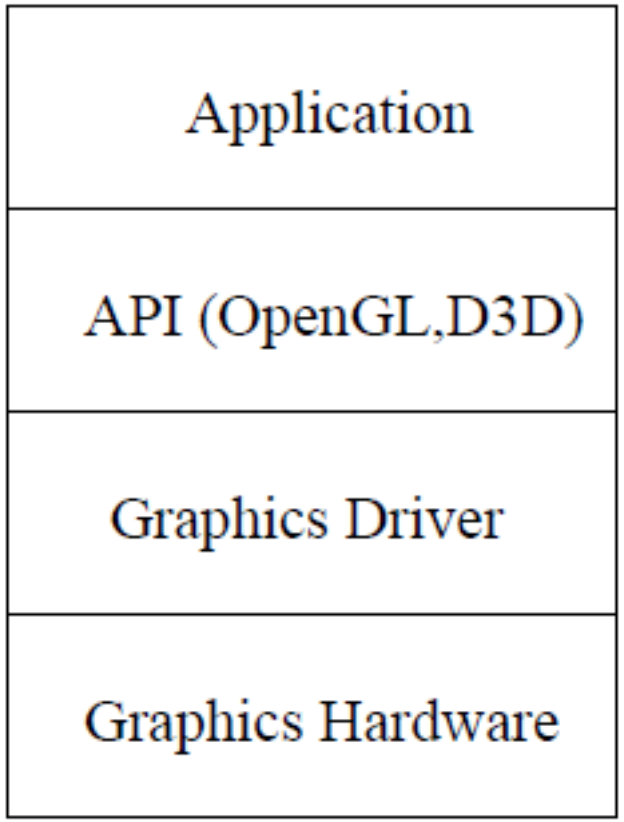
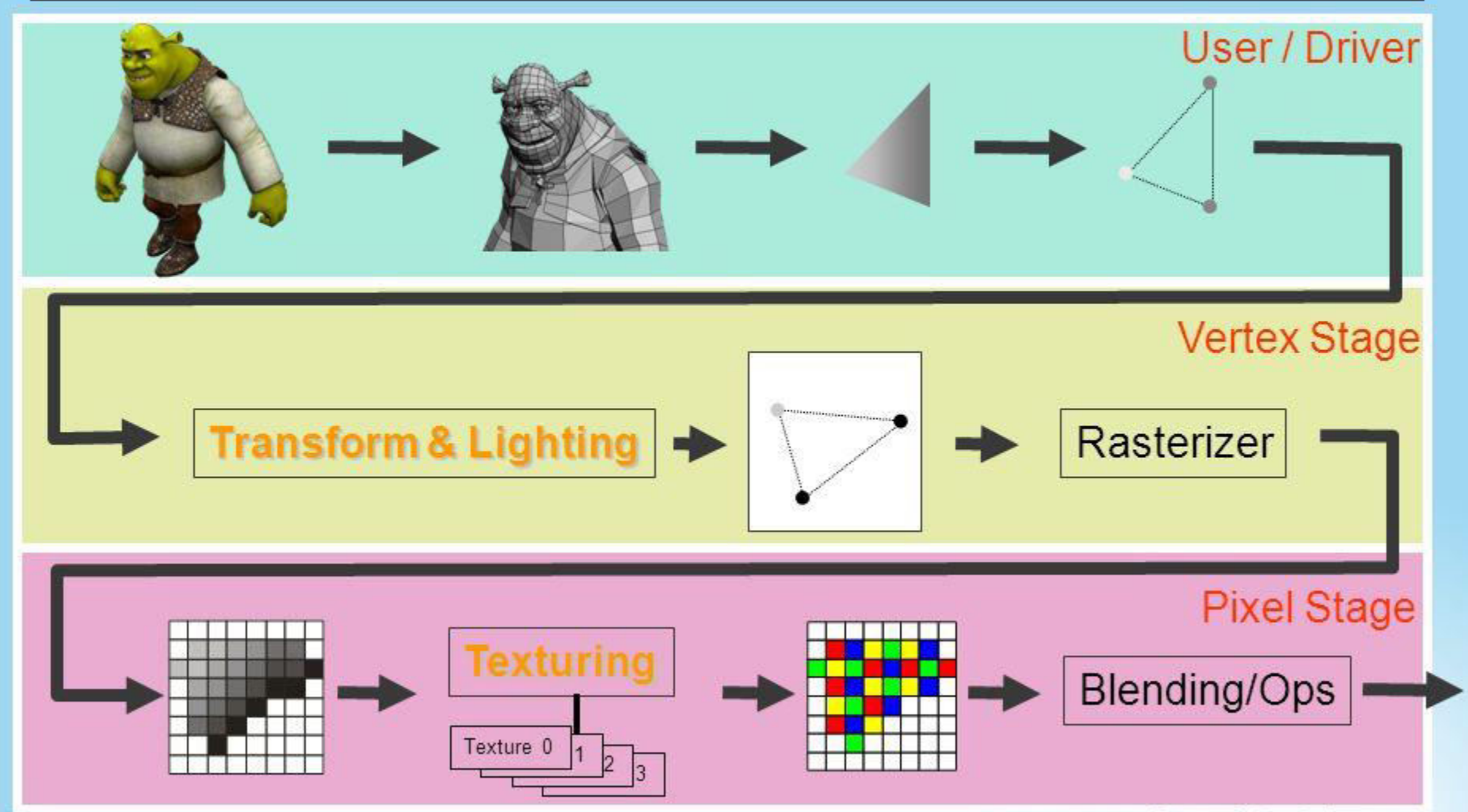


Figure 1: *Graphics hierarchy.*

From "A Crash Course on Programmable Graphics Hardware" by L. Wei

Overview


User / Driver
Vertex Stage
Transform & Lighting
Rasterizer
Pixel Stage
Texturing
Texture 0
1
2
3
Blending/Ops

Graphics Pipeline

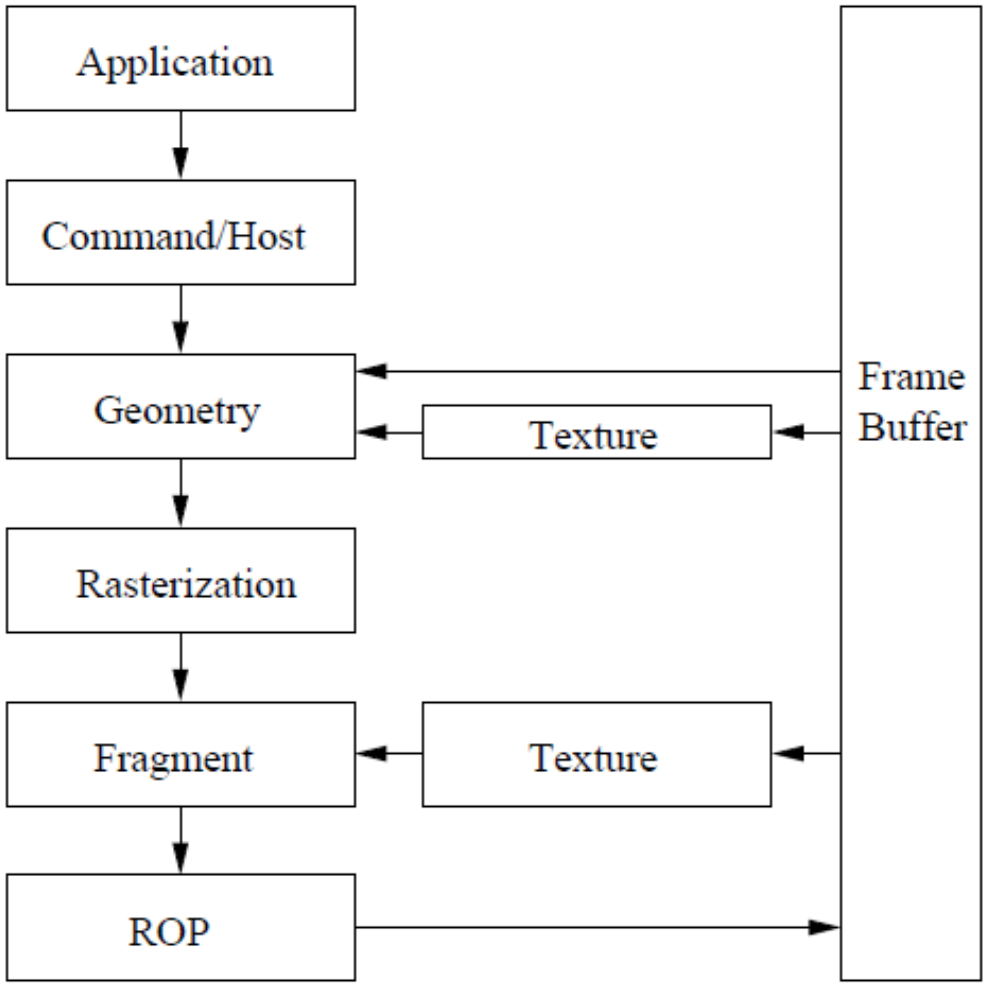
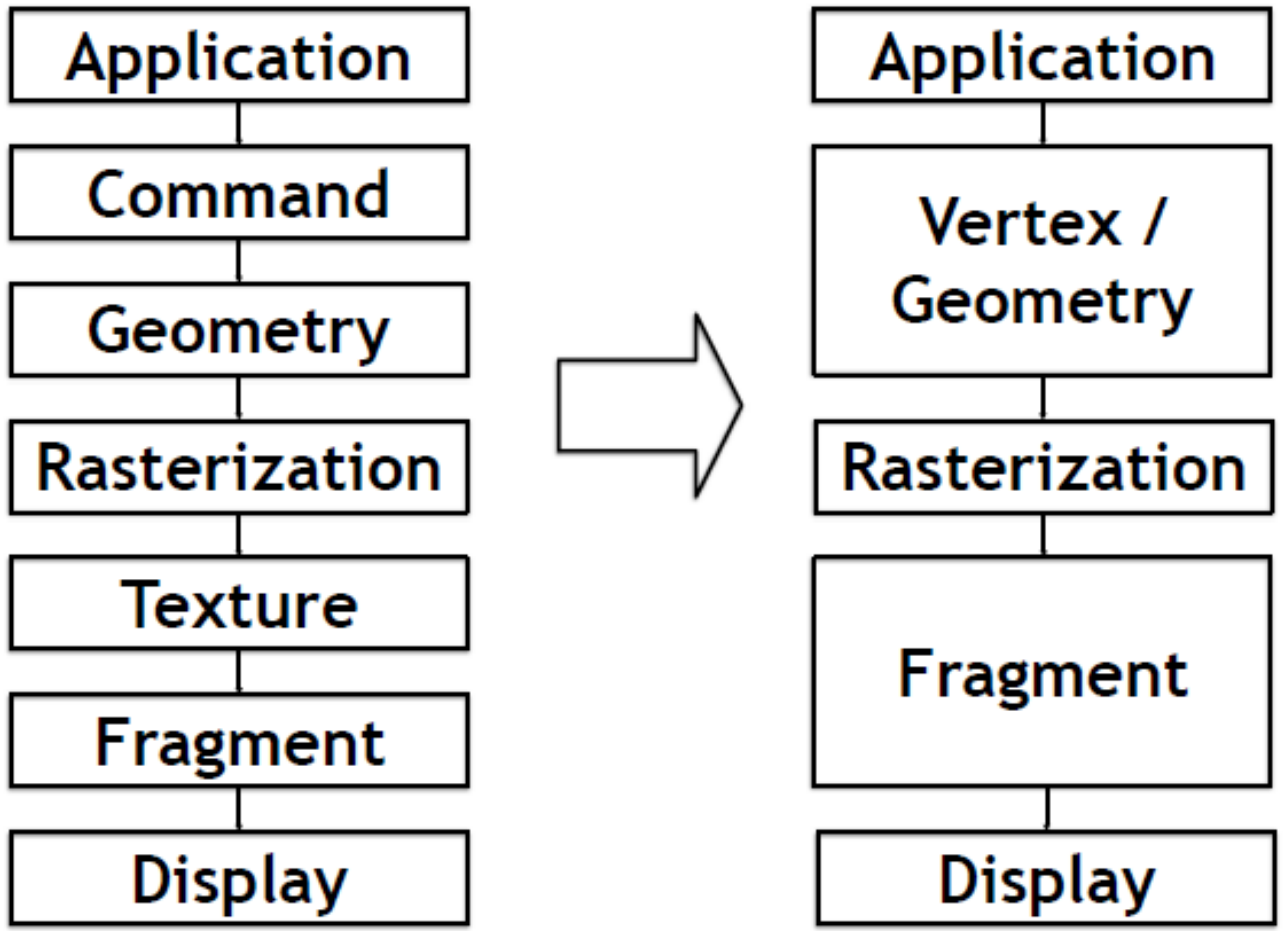


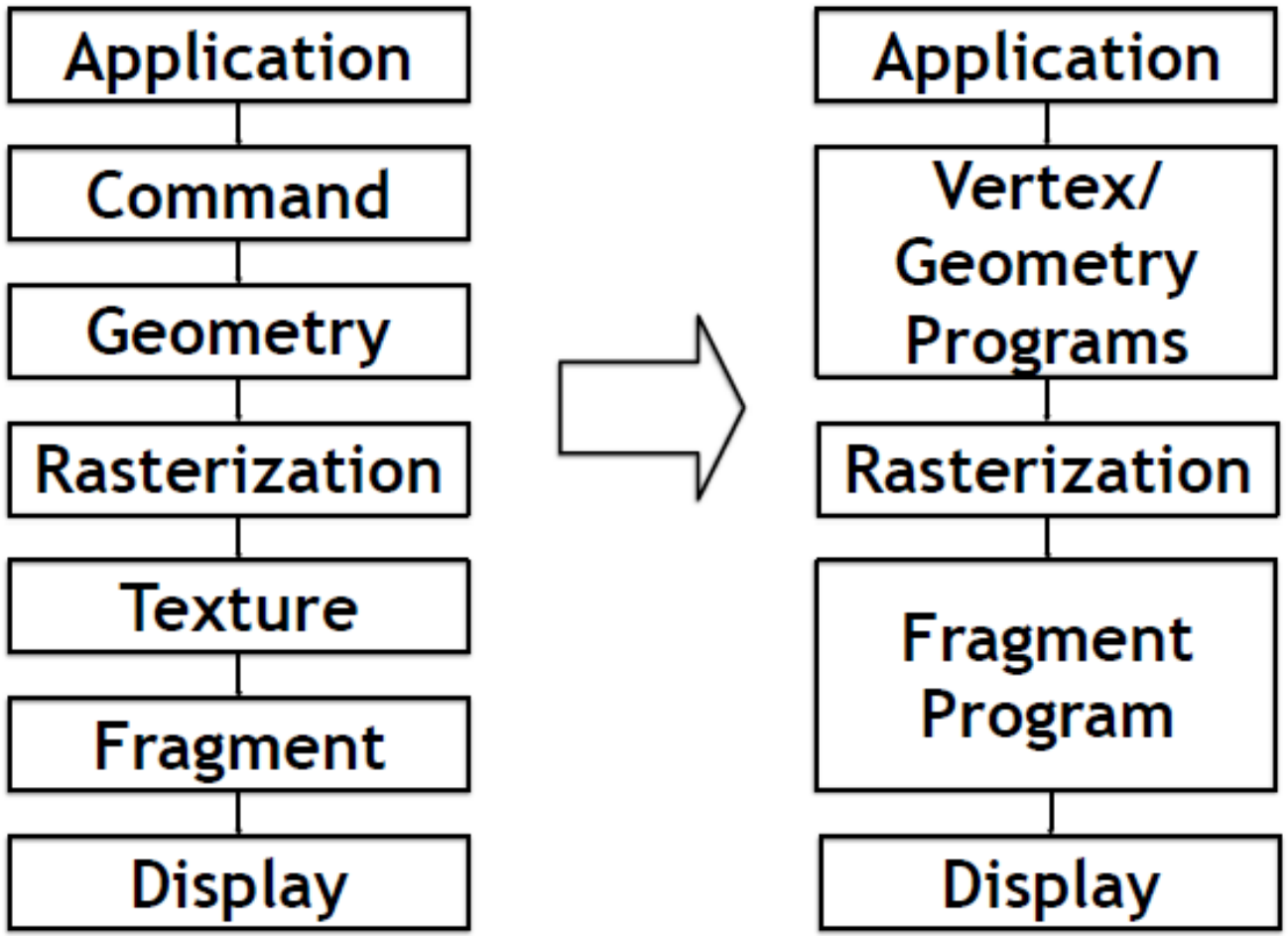
Figure 2: *The graphics pipeline.*

From “A Crash Course on Programmable Graphics Hardware” by L. Wei

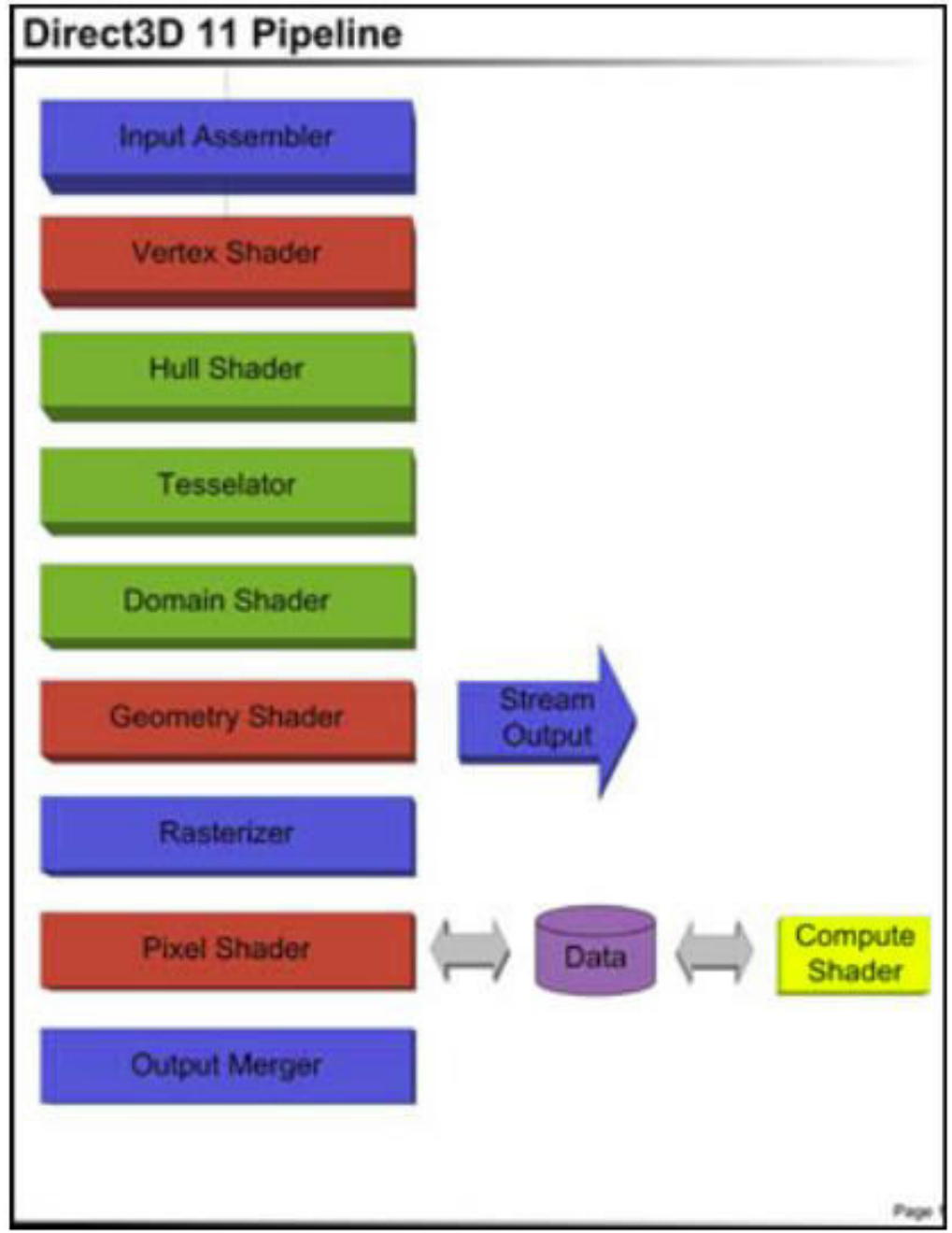
Graphics Pipeline



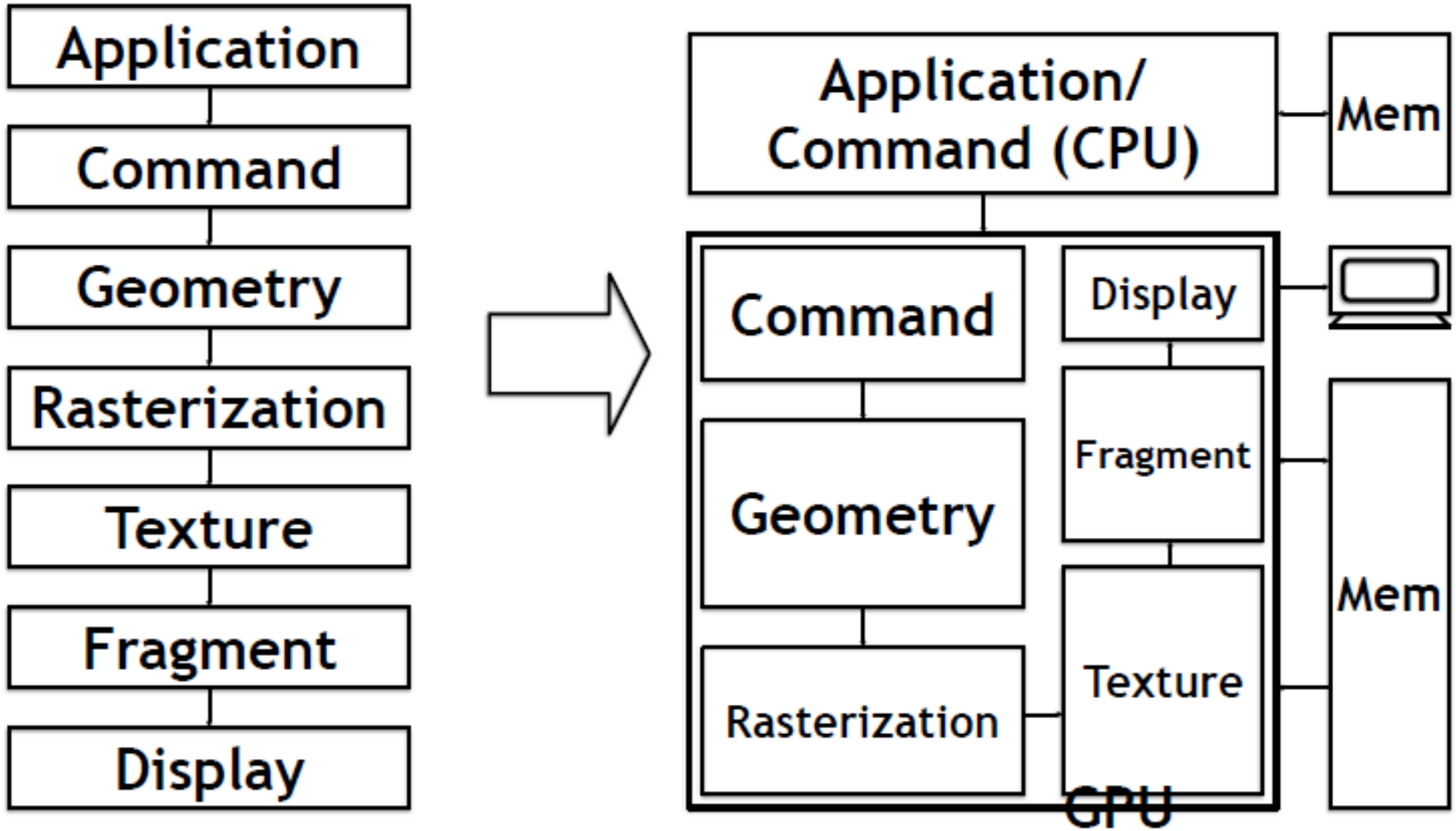
Programmable Graphics Pipeline



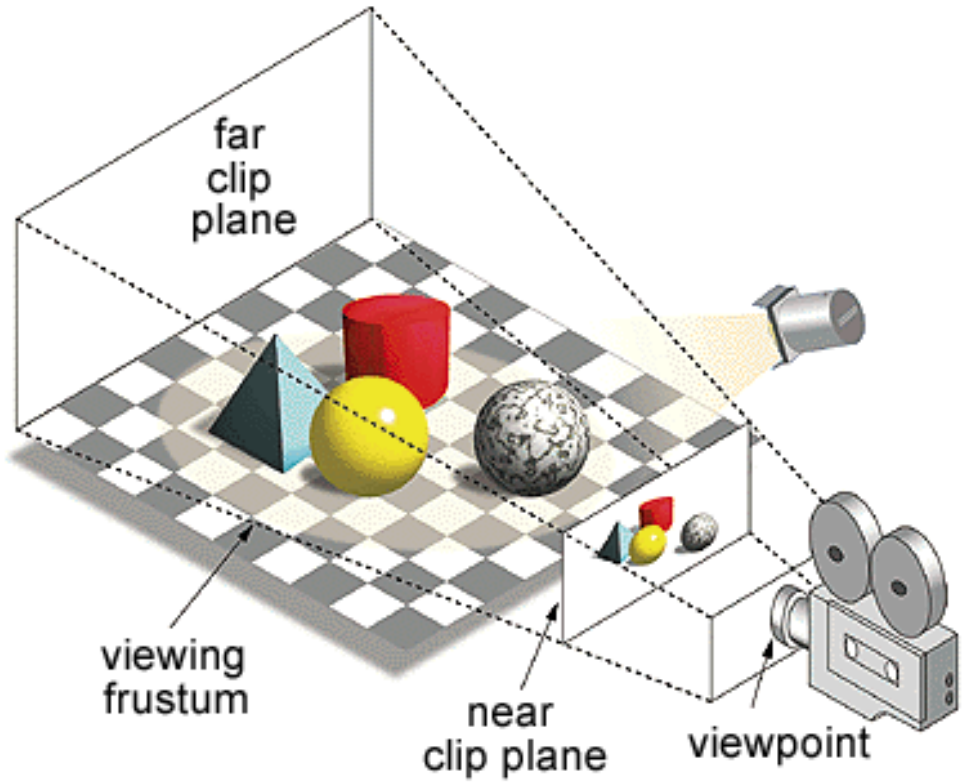
DX11

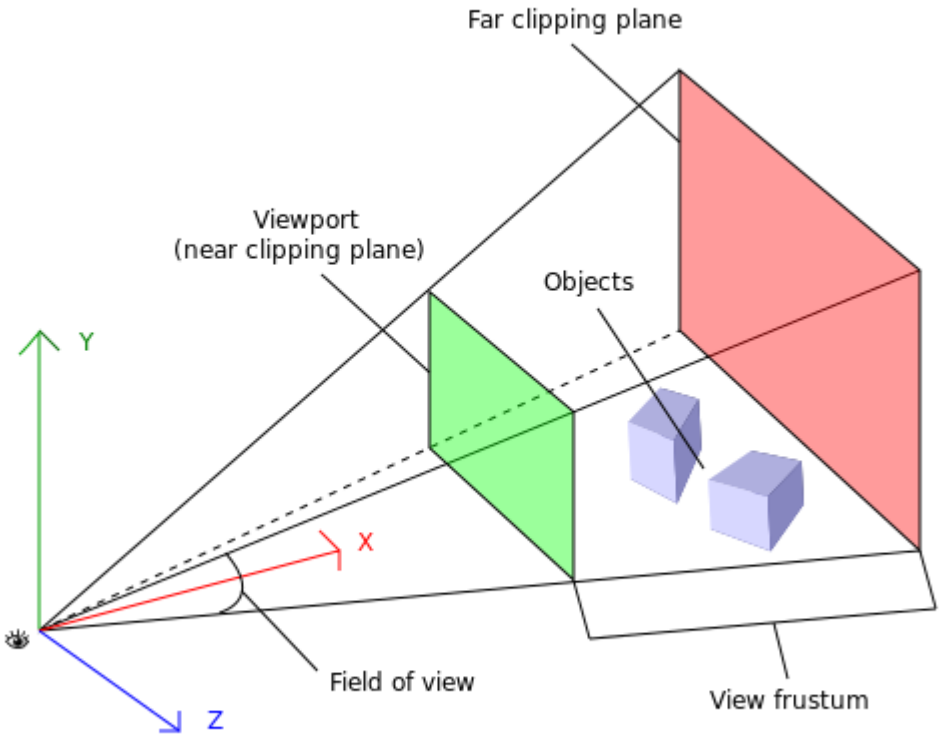

Direct3D 11 Pipeline
Input Assembler
Vertex Shader
Hull Shader
Tesselator
Domain Shader
Geometry Shader
Stream Output
Rasterizer
Pixel Shader
Data
Compute Shader
Output Merger

Graphics Hardware—Task Parallel



Geometry


far
clip
plane
viewing
frustum
near
clip plane
viewpoint


Far clipping plane
Viewport
(near clipping plane)
Objects
Y
X
Z
Field of view
View frustum

Homogeneous Coordinates

- Points and directions can be represented in homogeneous coordinates
- Think of homogeneous *w* as scaling term
- Why?
  - Allows more general xforms: 4-vector homogeneous transforms are much more general than 3-vector
  - Lazy evaluation: don't do divide until we need it

$$\begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix} \Leftrightarrow \begin{pmatrix} x/w \\ y/w \\ z/w \\ 1 \end{pmatrix}$$

About Matrices

- 4x4 matrix, 4-component coordinate
- Single-precision floating point is typical
- Matrices can be composed
  - By the application
  - By the implementation
- Affine (3x3)—preserves parallelism of lines
  - Translate, rotate, orthographic
- Perspective projection—not affine

$$\begin{pmatrix} x' \\ y' \\ z' \\ w' \end{pmatrix} = \begin{pmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \\ m_{41} & m_{42} & m_{42} & m_{44} \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix}$$

Homogeneous Transforms

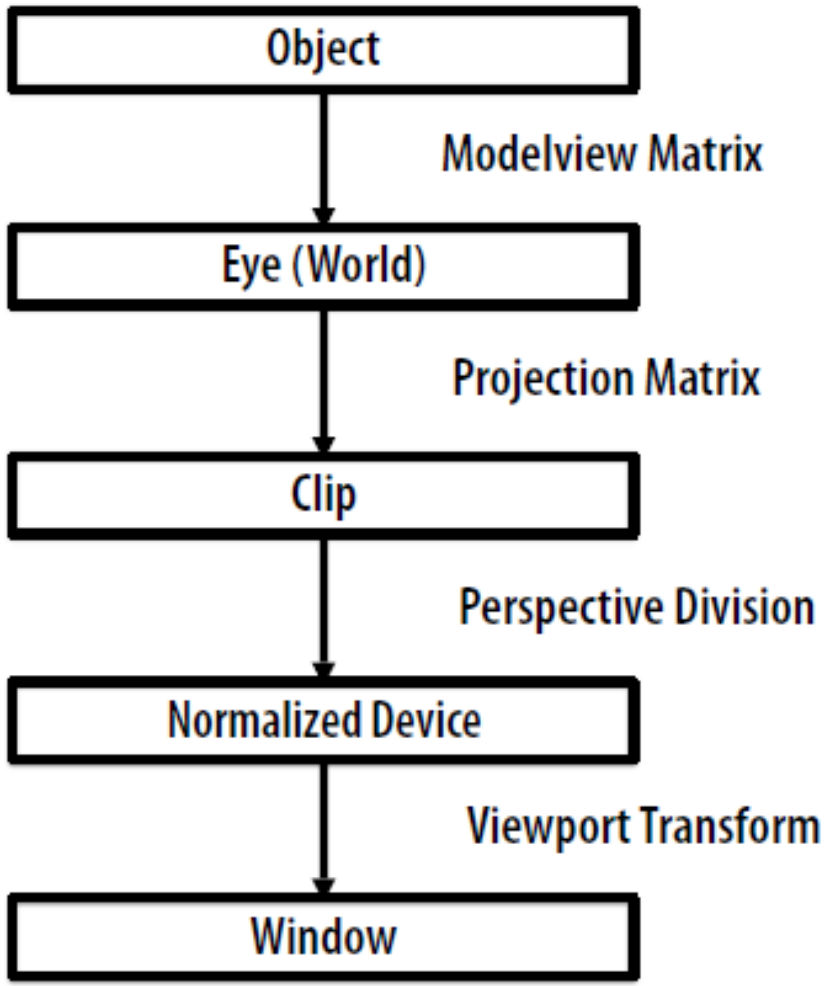
■ glFrustum

$$\begin{bmatrix} \frac{2 \cdot zNear}{right-left} & 0 & \frac{right+left}{right-left} & 0 \\ 0 & \frac{2 \cdot zNear}{top-bottom} & \frac{top+bottom}{top-bottom} & 0 \\ 0 & 0 & \frac{zFar+zNear}{zNear-zFar} & -\frac{2 \cdot zFar \cdot zNear}{zFar-zNear} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

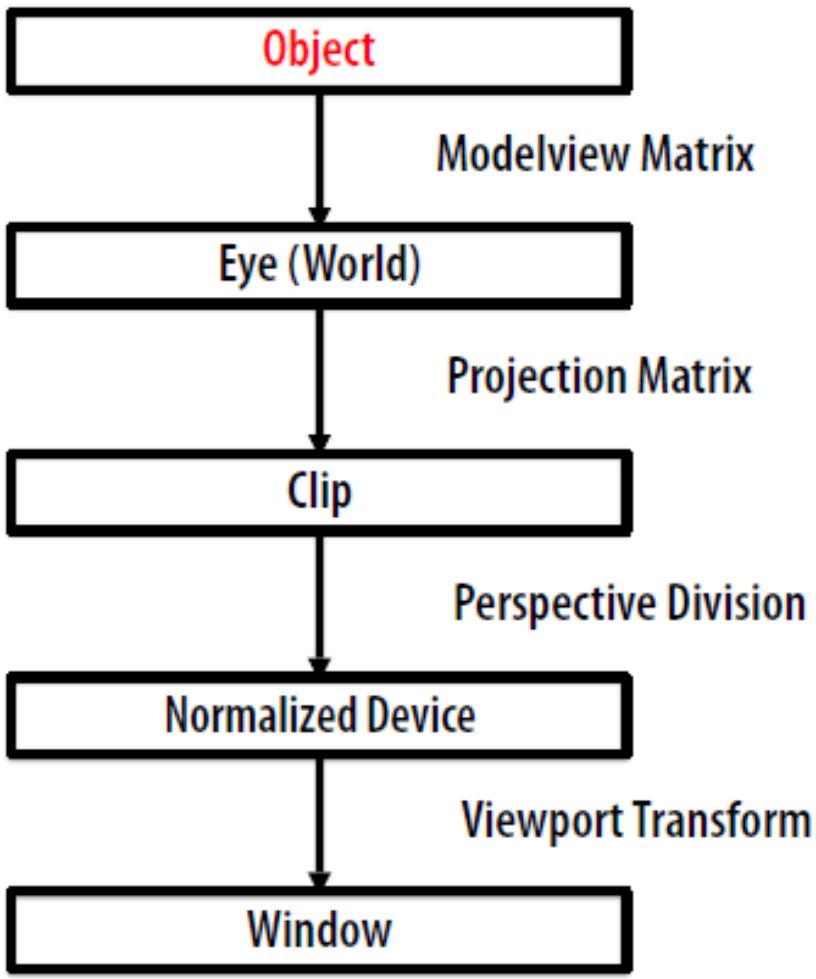
■ gluPerspective

$$\begin{bmatrix} \frac{f}{aspect} & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & \frac{zFar+zNear}{zNear-zFar} & \frac{2 \cdot zFar \cdot zNear}{zNear-zFar} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Coordinate System Transforms

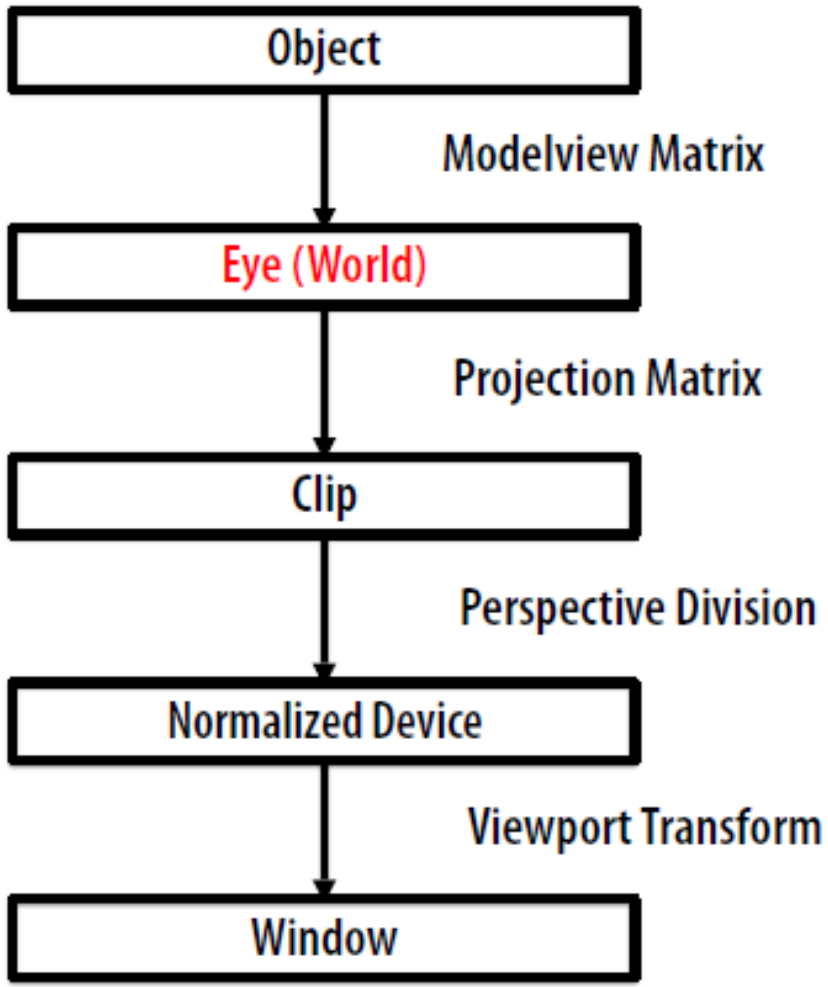


Coordinate System Transforms



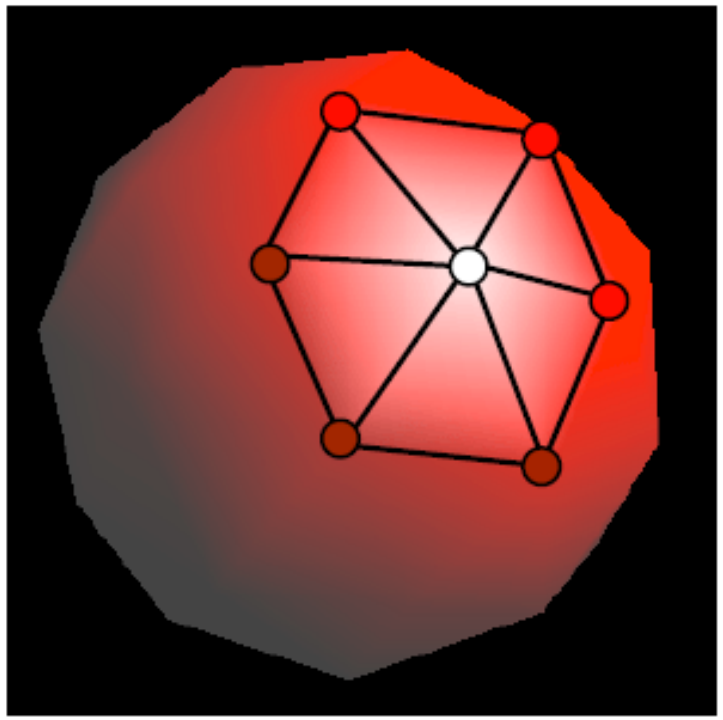
- Object space—natural coordinate system of models. Why? Easiest to model in this space.

Coordinate System Transforms

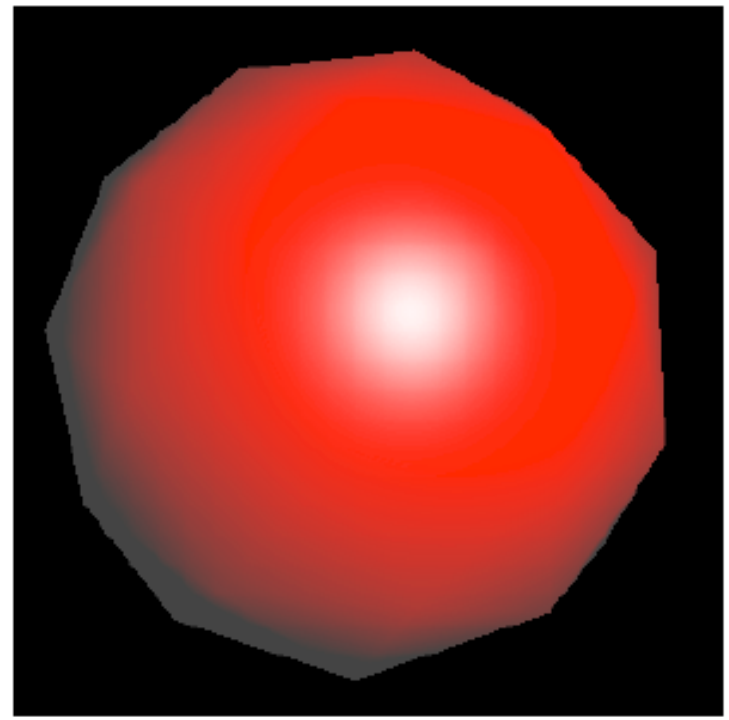


- Eye space—common space for all objects
- Why? Most suitable for lighting calculations
  - This is less used today (modern renderers back-project lights into object space and calculate there)
- User-specified clipping planes applied here

Example per-vertex computation: lighting



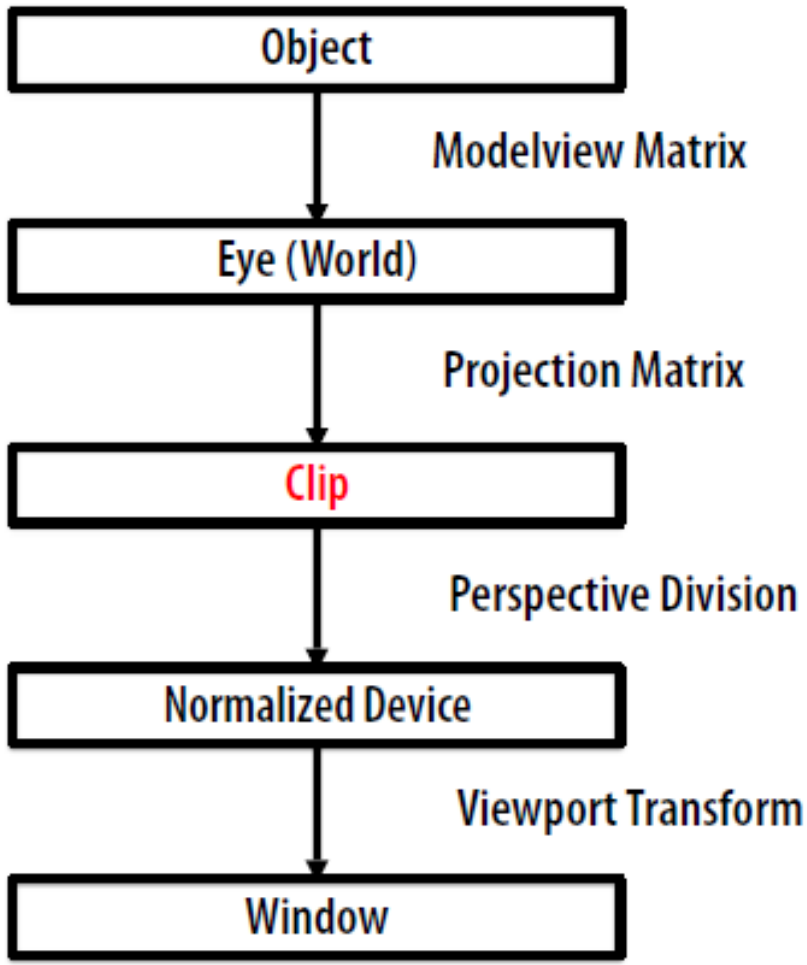
Per-vertex lighting computation



Per-vertex normal computation, per pixel lighting

- Per-vertex data: surface normal, surface color
- Uniform data: light direction, light color

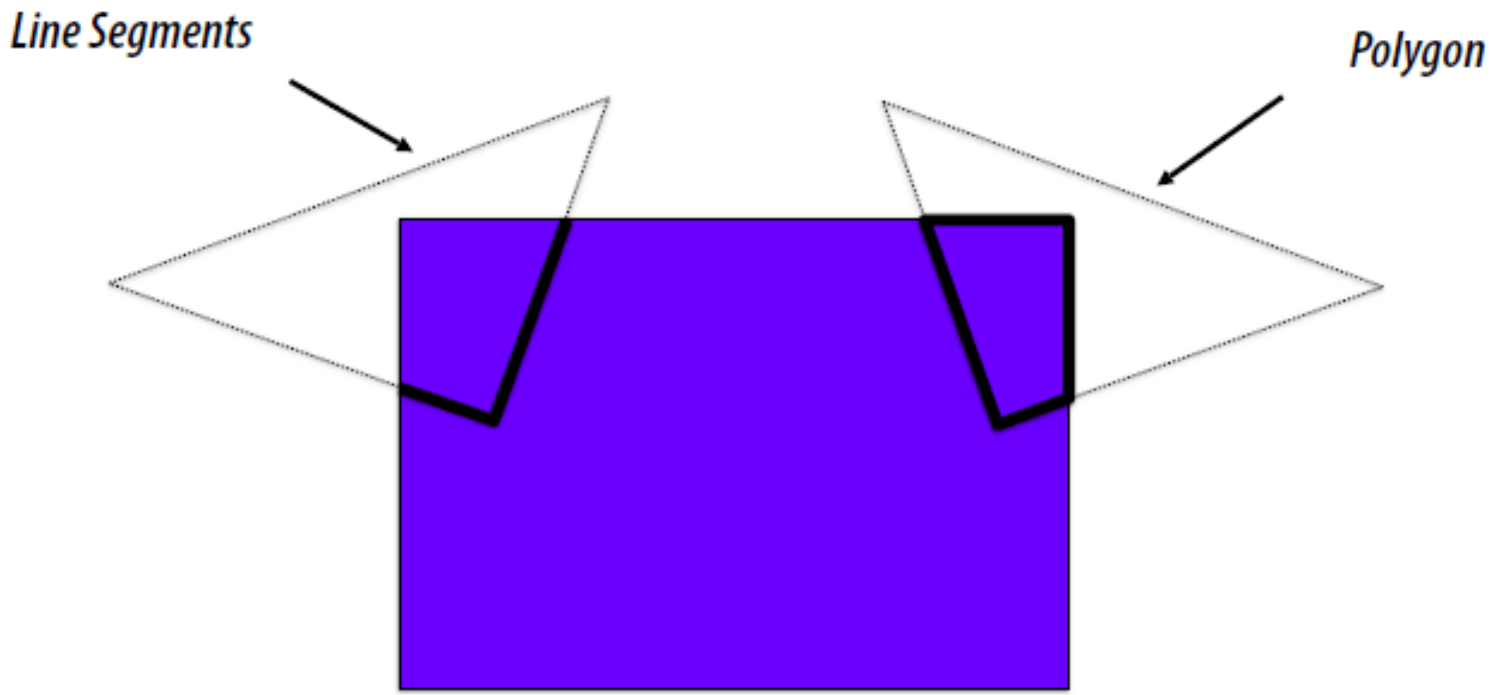
Coordinate System Transforms



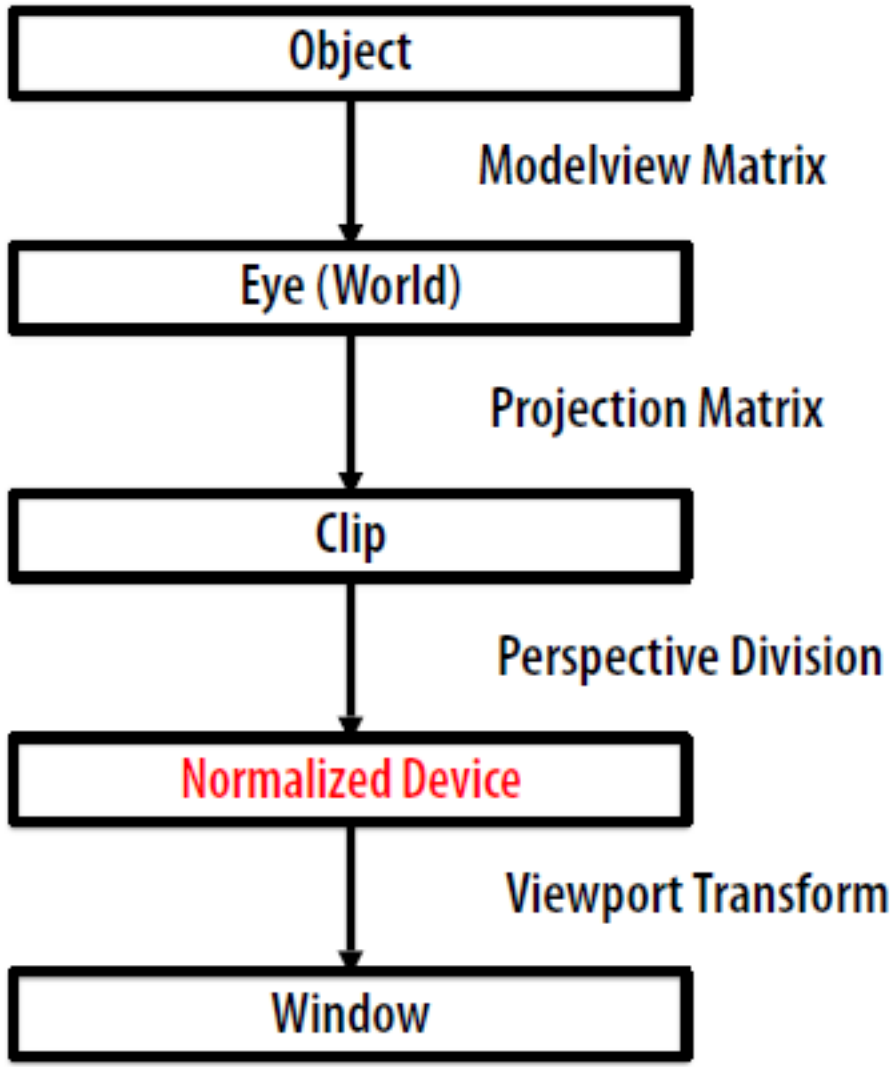
- Clip space—vertices are 4-vectors (have homogeneous *w*)
- Why? Clipping most efficient here—avoids perspective divide
- Vertex programs output positions in clip space
- Triangles assembled at this stage
- View volume clipping calculation against L/R/U/D/hither/yon: $\{x,y,z\}$ in $(-w, w)$

Clipping

- Two types
  - Point, line: eliminates geometry
  - Polygon: eliminates and introduces edges



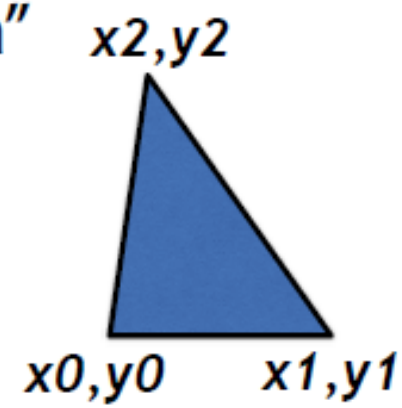
Coordinate System Transforms



- Normalized Device Coordinates
- $\{x,y,z\}$ in (-1, 1)
- Why? Culling most efficient here

Backface Cull

- Facet facing toward or away from viewpoint?
  - No facet normal (other APIs?)
  - Use sign of primitive's window coordinate "area"
    - Remember, only triangles are planar
- Use facing direction to
  - Select lighting result (for *n* or *–n*)
  - Potentially discard the primitive
- Advance in sequence to improve efficiency?



$$\text{Triangle area} = \frac{(x0y1 - x1y0) + (x1y2 - x2y1) + (x2y0 - x0y2)}{2}$$

Vertex Operations

Operations on vertices

- Vertex operations apply to vertexes independently

Transform coordinates and normal
- Model → world
- World → eye

Normalize the length of the normal

Compute vertex lighting

Transform texture coordinates

} vertex program

Transform to clip coordinates

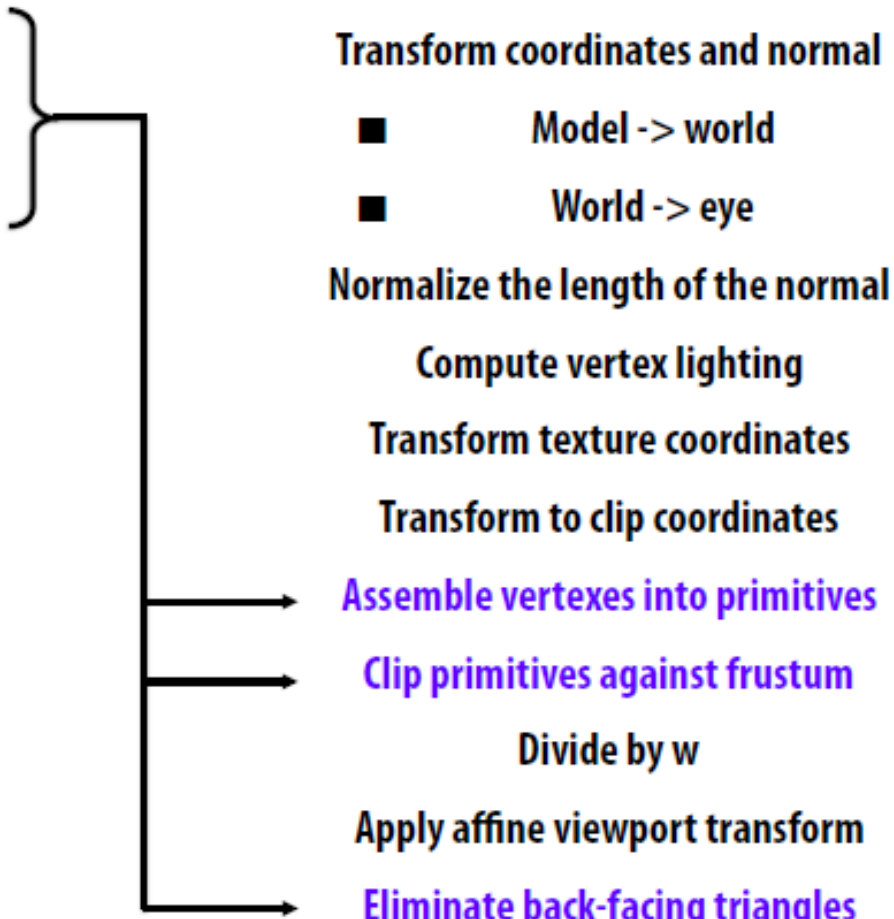
Divide by w

Apply affine viewport transform

Primitive Operations

Operations on primitives such as polygons/triangles

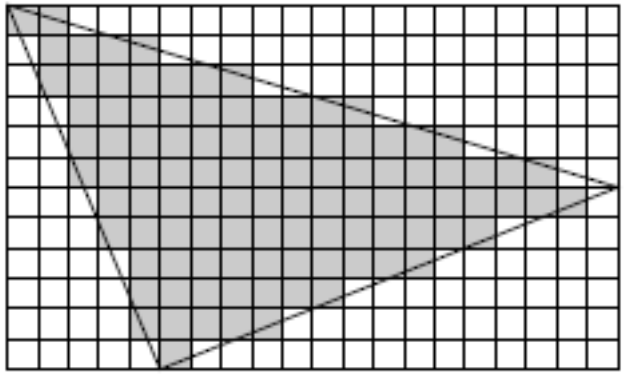
- Primitive assembly
- Clipping
- Backface cull



Rasterization

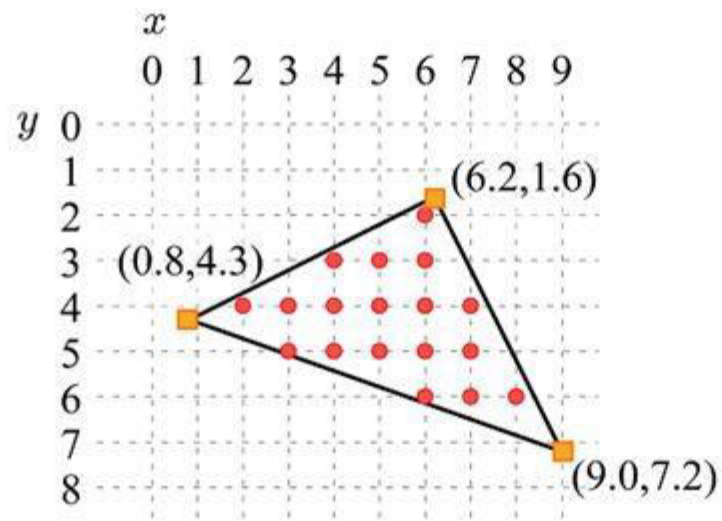
- Goal: Convert a triangle to a set of covered screen pixels
- Two steps
  - 1. Determine which pixels are part of the triangle
  - 2. Compute the attributes (color, normal, texture coordinates, etc.), of pixels by interpolating from vertices.

$$\bar{C}_p = \sum_{i=A,B,C} w_i . \bar{C}_i$$



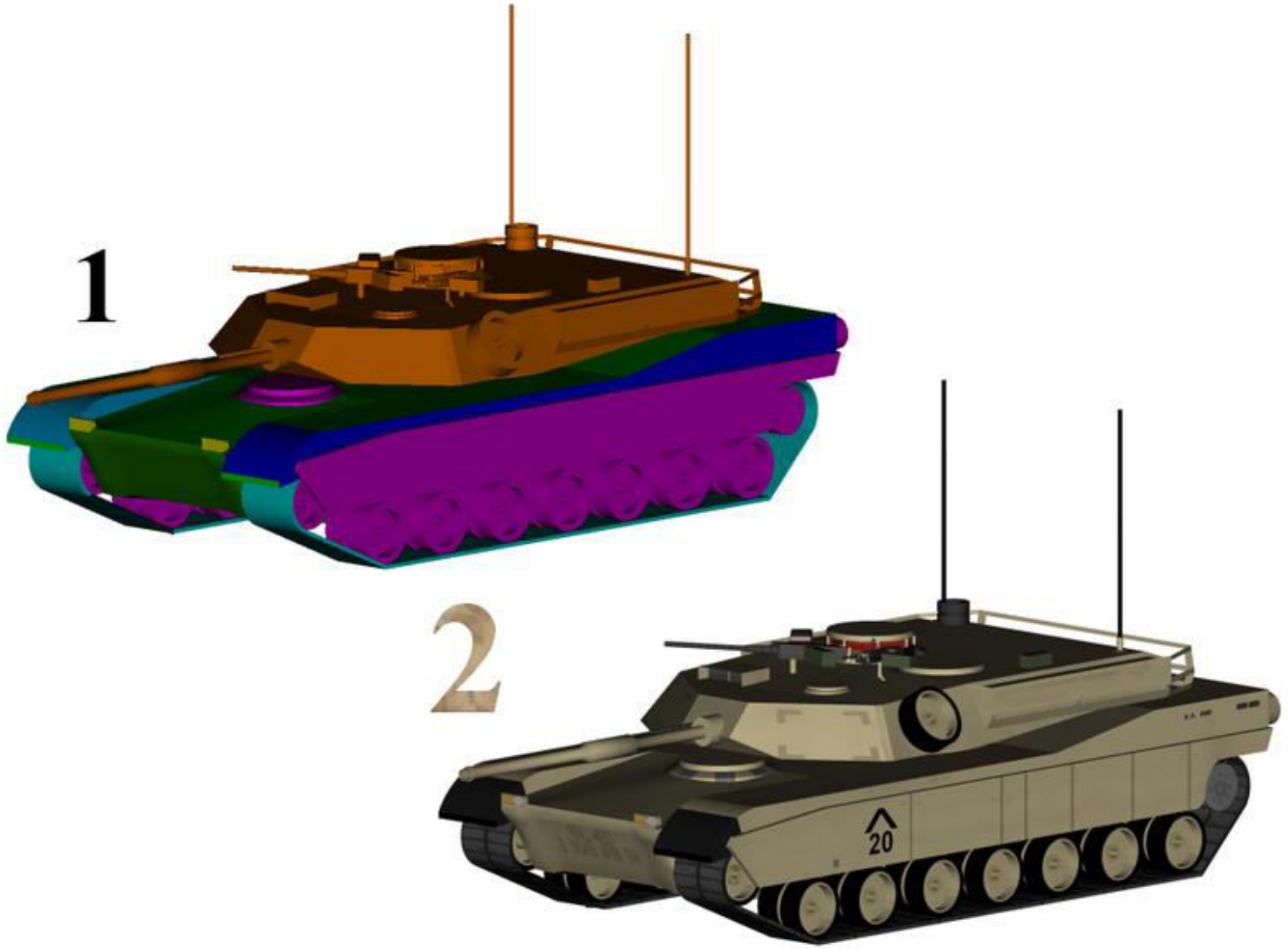
Fragment Processing

- The per-fragment attributes may include a normal vector, a set of texture coordinates, a set of color values, a depth, etc.

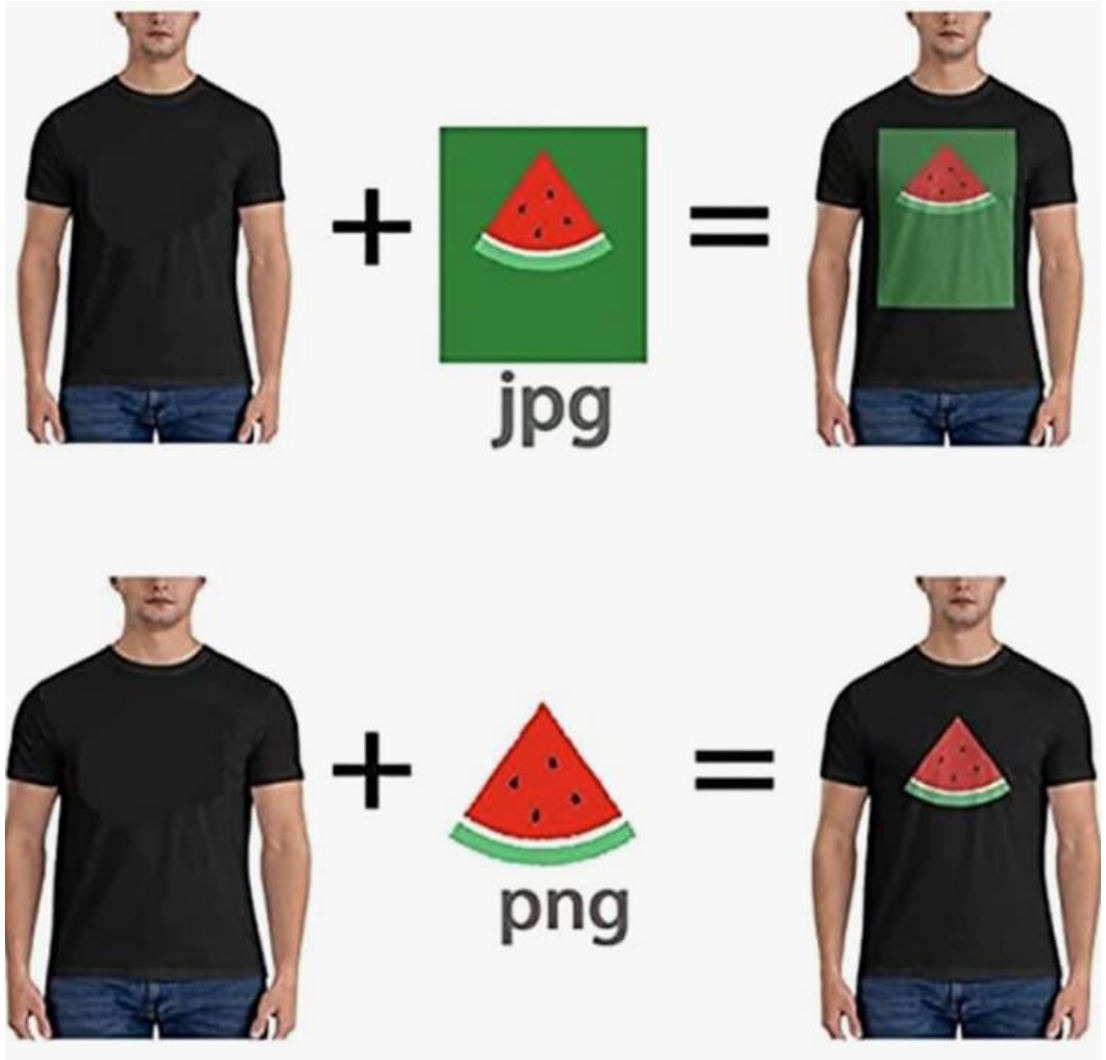


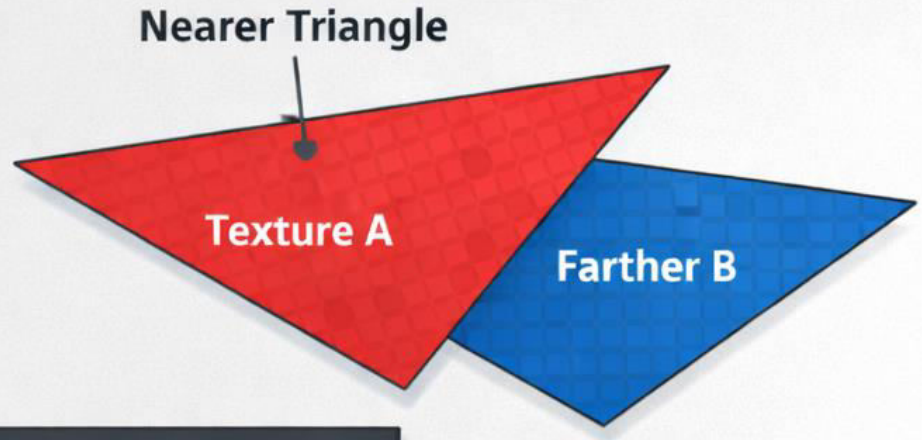
- Using these data, the fragment processing stage determines the final color of each fragment.
- Two most important things to do in the fragment processing stage
  - Per-fragment lighting (presented in Chapter 5)
  - Texturing (presented in Chapters 8, 9, and 10).
- Let's see what texturing is.

Texture (from Wikipedia)



Texture





Fragment A (Near)

Depth: 0.3

Color: ***TexA(uvA)***

Fragment B (Far)

Depth: 0.7

Color: ***TexB(uvB)***

Z Test

```
Fragment Shader
 vec4 color = texture(textureSampler, uv);
 float depth = gl_FragCoord.z;
```

ROP

Render Output Unit

Frag A

Frag B

Key Functions:

- Depth Test *(Z-Buffer)*
- Blending
- Write to *Framebuffer*

Z Pass: 0.3 (Closer)

Final Color: Tex*A(uvA)*

Discarded

What is a pixel?

- "A Pixel Is Not A Little Square, A Pixel Is Not A Little Square, A Pixel Is Not A Little Square! (And a Voxel is Not a Little Cube)", Alvy Ray Smith, Microsoft Tech Memo 6
- A pixel is a point sample
- An image is a grid of point samples
- . . . can be reconstructed into a continuous function by
  - The Sampling Theorem [Shannon]
  - Your eye
- A pixel represents the contributions of an area in screen space—area integral
- Point sampling = approximation of that area with single point

Display

- **Antialiasing**
- **Compositing**
- **Depth Buffer**
- **Monitors**


02:36
record
00:04
time
1
2
3
FSAA Off
02:36
record
00:14
time
4X FSAA

Compositing



Foreground = FrgdGrass **over** *Rock* **over** *Fence* **over** *Shadow* **over** *BkgdGrass*



Hillside = ***Plant over*** *GlossyRoad* **over** *Hill;*



Background = Rainbow **plus** *Darkbow* **over** *Mountains* **over** *Sky;*

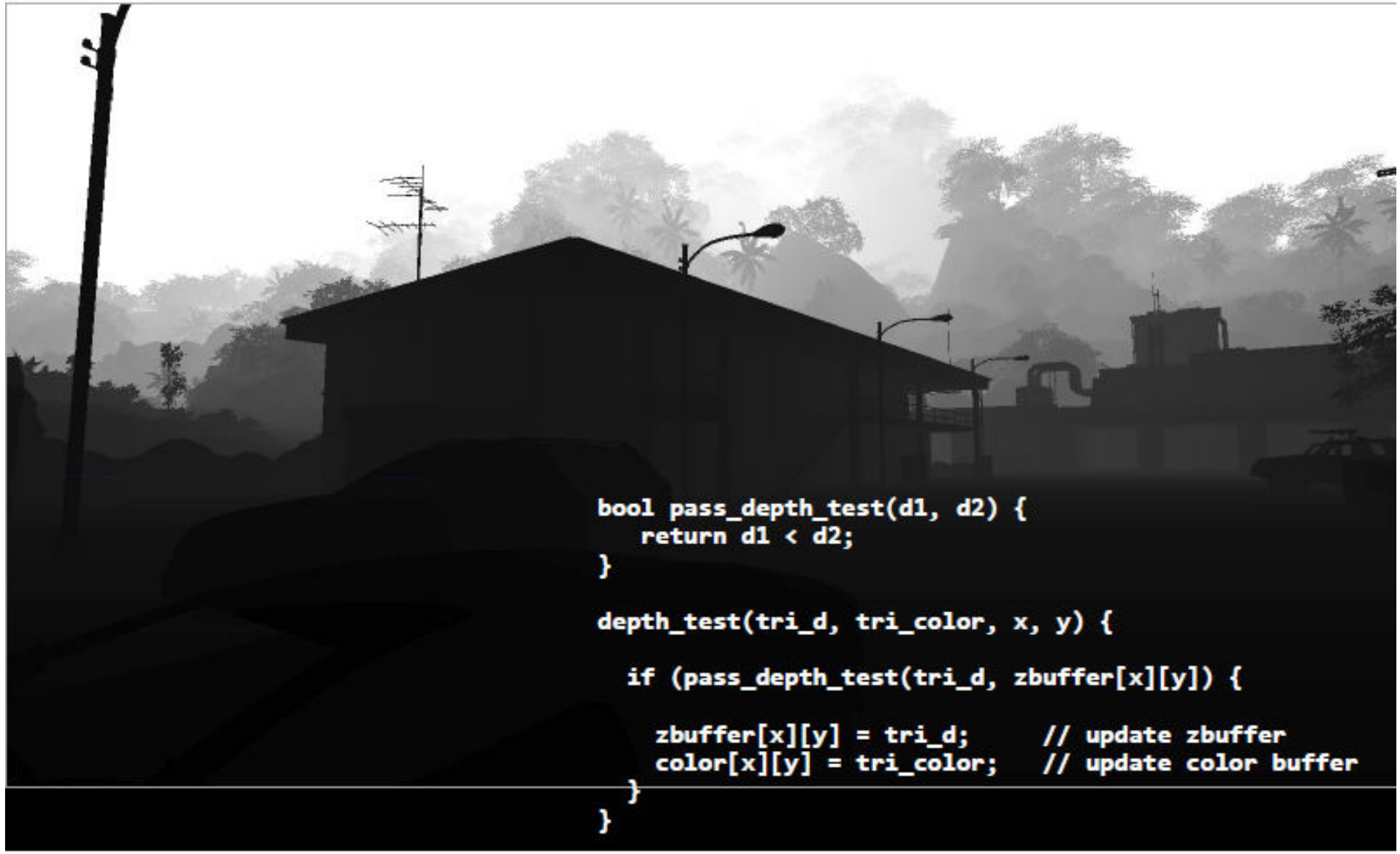


Pt.Reyes = Foreground **over** *Hillside* **over** *Background.*



[Lucasfilm, "Road to Point Reyes"]

Occlusion via the depth buffer



```
bool pass_depth_test(d1, d2) {
   return d1 < d2;
}

depth_test(tri_d, tri_color, x, y) {

  if (pass_depth_test(tri_d, zbuffer[x][y]) {

    zbuffer[x][y] = tri_d;      // update zbuffer
    color[x][y] = tri_color;    // update color buffer
  }
}
```

Programmable shaders

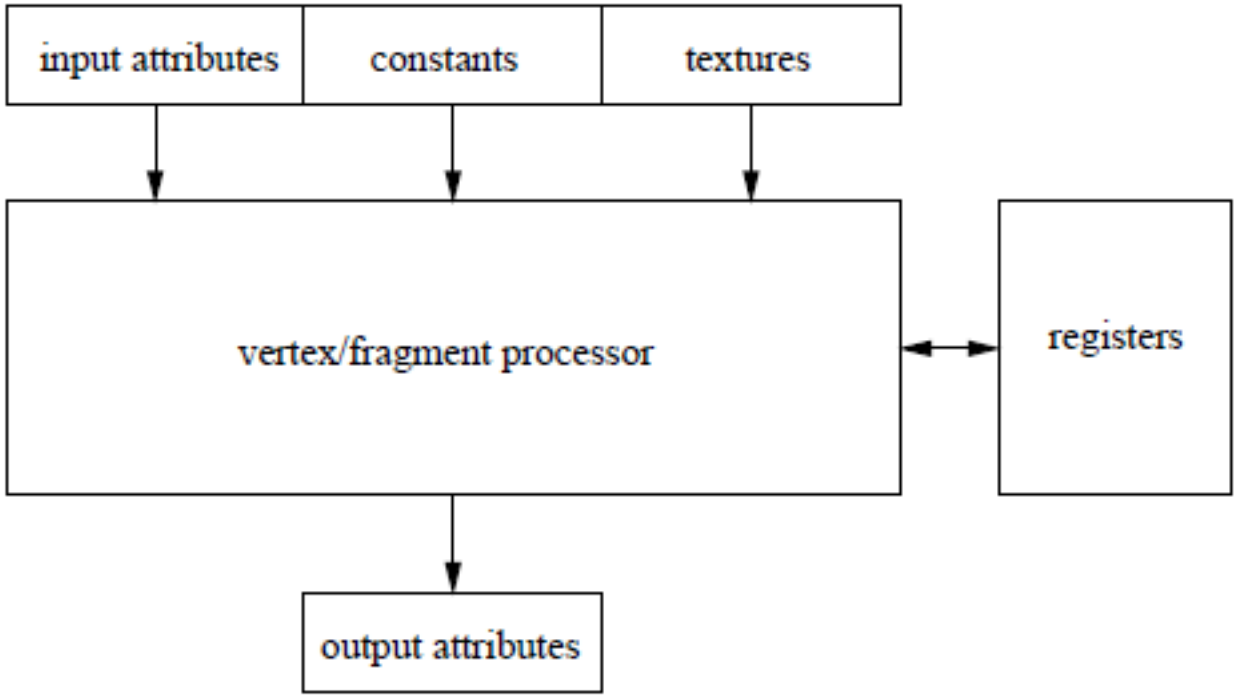


Figure 6: *Conceptual model of a programmable vertex/fragment processor.*

High Level Shading Language (HLSL)

```
#the following are constant inputs to the program
float4x4 T; # transformation matrix
float3 light; # light direction
#the following are per vertex attributes
float4 input_position;
float3 input_normal;
float4 input_color;
# compute output position
float4 output_position = T*input_position;
# compute output color
float4 output_color = dot(light,input_normal)*input_color;
```